



CINEMATCH

An AI-Powered Movie & Game Recommender

Describe your mood — get the perfect pick, and the reason why.

Software Engineering Capstone · Sejong University

Asliddin Ismail · Chae Hunwoo

Too much choice, too little time

“

*People often spend **longer choosing** what to watch than they spend enjoying a short film.*

Decision fatigue is real.



Endless scrolling

Streaming menus offer hundreds of rows and no clear answer.



Optimized for engagement

Platforms maximise watch-time, not a confident decision.



No reason given

Recommendations rarely explain why a title suits you.

One answer. And the reason why.

CINEMATCH is a web app where you describe your mood in plain words — and an **AI** plus a **C engine** instantly pick the perfect movie or game for tonight, and tell you why.

Opinionated

One confident pick — not 20 rows of thumbnails.

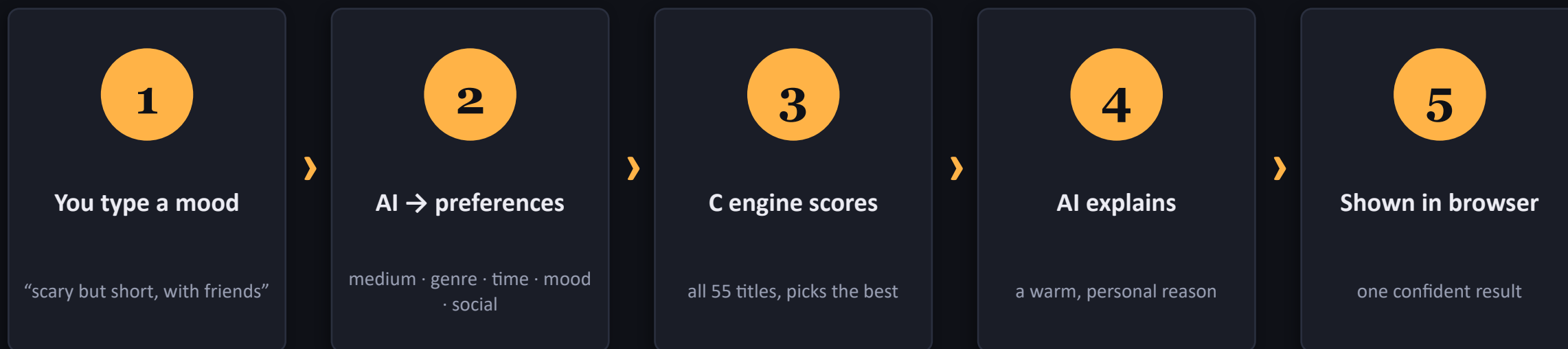
Explained

A warm, plain-language reason you can trust.

Hybrid

AI for language, C for deterministic logic.

The five-step pipeline



AI handles language · C handles deterministic logic

Domain-Driven Design

Separate the business idea from the plumbing — each layer has one job.

Interfaces

What users reach: web pages + HTTP API

Vite + React · Express

Application

Use cases: understand → pick → explain → discuss

Node.js

Domain

Core concepts & rules · the scoring

Node.js + C engine

Infrastructure

The outside world: AI client, engine runner

OpenRouter · child process

The Node backend runs the compiled C program as a child process, exchanging JSON.

An LLM at three points



Mood understanding

Reads free text and emits structured preferences as strict JSON — turning vague human language into machine-usable data.



Explanation generation

After the C engine picks a title, the LLM writes a warm two-to-three sentence reason, tailored to what the user said.



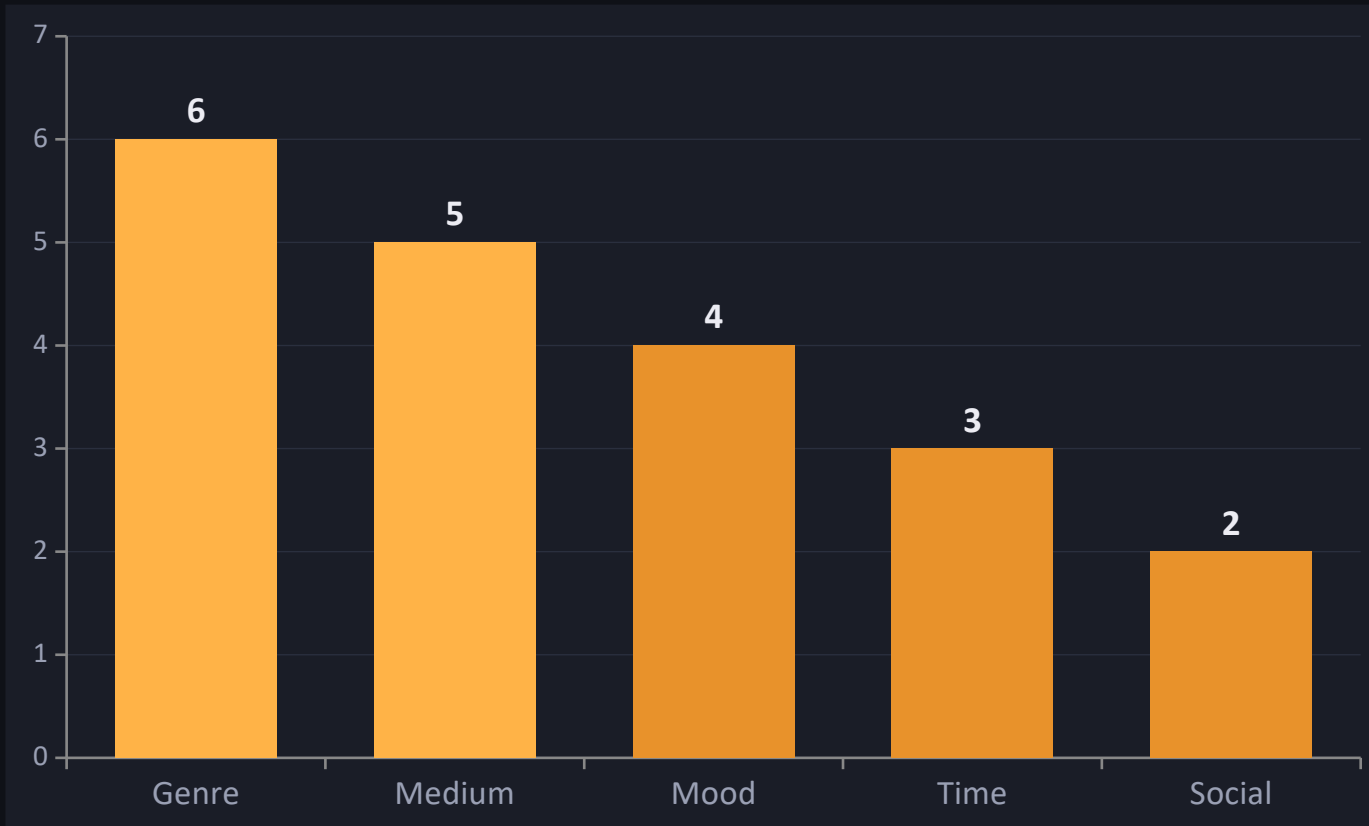
Movie discussion

Generates a full analysis of any film: theme, epic moments, director's vision, and discussion questions.

Provider: **OpenRouter — free open models (Qwen3)** · zero cost for students

Transparent, rule-based scoring

Each title earns points; the highest score wins. The biggest weights reflect what matters most.



Engine Mode

One C program, two modes:

No args

→ interactive 5-question quiz

5 args

→ prints one line of JSON

```
cinematch Movie horror short  
intense group
```

Modeling the library with structs

```
typedef struct {
    const char *type;    /* Movie or Game */
    const char *title;
    int year;
    const char *genre;
    const char *mood;
    const char *timeNeed;
    const char *social;
    const char *rating;
    const char *desc;
} Title;

/* The fixed library of 55 titles */
Title library[] = {
    {"Movie", "Interstellar", 2014, "scifi",
     "deep", "medium", "any", "8.7", ... },
    /* ... 54 more titles ... */
};

int libSize =
    sizeof(library)/sizeof(library[0]);
```

C techniques here

- typedef struct — a custom data type
- Array of structs — the whole library
- const char * — pointers to strings
- sizeof trick — array length, computed at compile time
- One source of truth: titles.h

The scoring algorithm

```
● ● ●  
  
#define PTS_GENRE    6  
#define PTS_MOOD     4  
  
/* How well one title fits the answers */  
static double scoreTitle(const Title *t,  
    const char *genre,  
    const char *mood, ...) {  
    double score = 0.0;  
    if (strcmp(t->genre, genre) == 0)  
        score += PTS_GENRE;  
    if (strcmp(t->mood, mood) == 0)  
        score += PTS_MOOD;  
    /* rating breaks ties: 8.7 -> +0.87 */  
    score += atof(t->rating) / 10.0;  
    return score;  
}
```

C techniques here

- static — file-private function (encapsulation)
- const Title * — pointer to a read-only struct
- strcmp() — string compare, <string.h>
- atof() — text to double, <stdlib.h>
- #define — compile-time constants

One program, two modes — the C↔JS bridge

```
int main(int argc, char **argv) {
    /* Engine mode: Node runs the program as
       cinematch <medium> <genre> <time>
       <mood> <social> */
    if (argc == 6) {
        recommendJson(argv[1], argv[2],
                      argv[3], argv[4],
                      argv[5]);

        return 0;
    }
    /* else: ask 5 questions, then
       recommend(...) prints to screen */
}
```

C techniques here

- argc / argv — command-line arguments
- Separate compilation — recommend.h interface
- putchar / fputs — character output
- Exit codes — return 0

C prints to stdout: `{"found":true,"title":"Get Out", ... }` → Node.js: `JSON.parse(stdout)`

Four pages, one app



AI Match

The core feature. Type a mood, get one explained pick from the curated 55-title library.



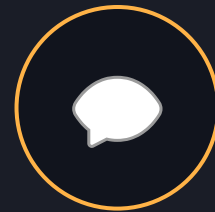
Movies

Browse thousands of real films from the open TMDB dataset — search, filter, posters, details.



Games

Browse 800,000+ titles from the open RAWG dataset — covers, ratings, and detail views.

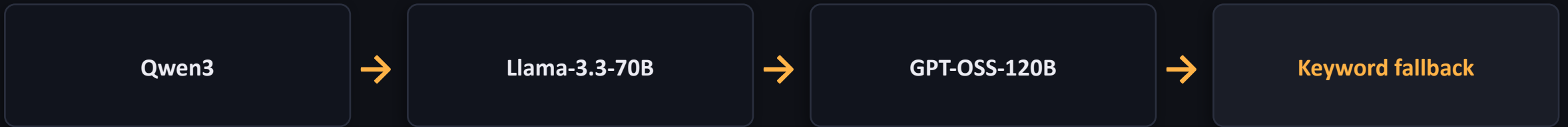


Discussion

Search any film for AI analysis: cast, theme, epic moments, and discussion questions.

Never crash in a live demo

Free AI models rate-limit and fail. CINEMATCH degrades gracefully at every layer.



Automatic retry

On a 429 or 5xx error, the request retries after a short delay.



Keyword fallback

If all AI fails, a local matcher still derives sensible preferences.



Content fallback

Empty AI fields are replaced with rich, well-written defaults.

Rehearsed and reliable

“relaxing cozy solo night”



Balatro (2024)

Cozy · Relaxed · ★ 9.0

“scary horror with a group”



Alien (1979)

Horror · Intense · ★ 8.5

“epic action game for hours”



Super Mario Odyssey

Action · Light · ★ 9.4

Same mood → same pick, every time. The C engine is deterministic.

Testing the recommendation pipeline

Mood input	Expected preferences	Actual output	Result
"relaxing cozy solo night"	Cozy · Relaxed · solo	🎮 Balatro (2024)	✓ Pass
"scary horror with a group"	Horror · Intense · group	🎬 Alien (1979)	✓ Pass
"epic action game for hours"	Action · long · solo	🎮 Super Mario Odyssey	✓ Pass
"thoughtful sci-fi, alone"	Sci-Fi · deep · solo	🎬 Arrival (2016)	✓ Pass

Edge cases handled



Mixed mood

"happy but tense" → the AI picks the dominant signal; a valid pick still returns.



AI failure

Rate-limit or error → keyword fallback derives preferences. Never crashes.



Empty input

Blank text is rejected with a friendly message before any work runs.



Gibberish

Unrecognised text falls back to safe default preferences — always a result.

Tech stack & open data

ENGINE

C → cinematch.exe

BACKEND

Node.js + Express

FRONTEND

Vite + React

PACKAGES

pnpm

AI

OpenRouter (Qwen3)

MOVIES

TMDB open dataset

GAMES

RAWG open dataset

VERSION CONTROL

Git + GitHub

Limitations & future work

Current limitations

- No user accounts — the app is stateless
- Recommends only from the 55-title curated library
- Runs locally — not yet deployed publicly
- Free AI models can be slow or rate-limited

Future work

1. User profiles & saved watch history
2. Personalization that learns from your feedback
3. Larger library — recommend from TMDB & RAWG directly
4. Deployment: web hosting + database integration
5. Automated tests (TDD) for the domain layer



Classic C meets modern AI.

A focused idea, executed cleanly — deterministic logic in C, language in an LLM,
wrapped in a polished, well-architected product.

Thank you

github.com/zubaydullayevasliddin06-cmd · CINEMATCH